

Taller Herramienta Python Tutor

Diego Javier Torres Auz

Universidad Técnica Particular de Loja

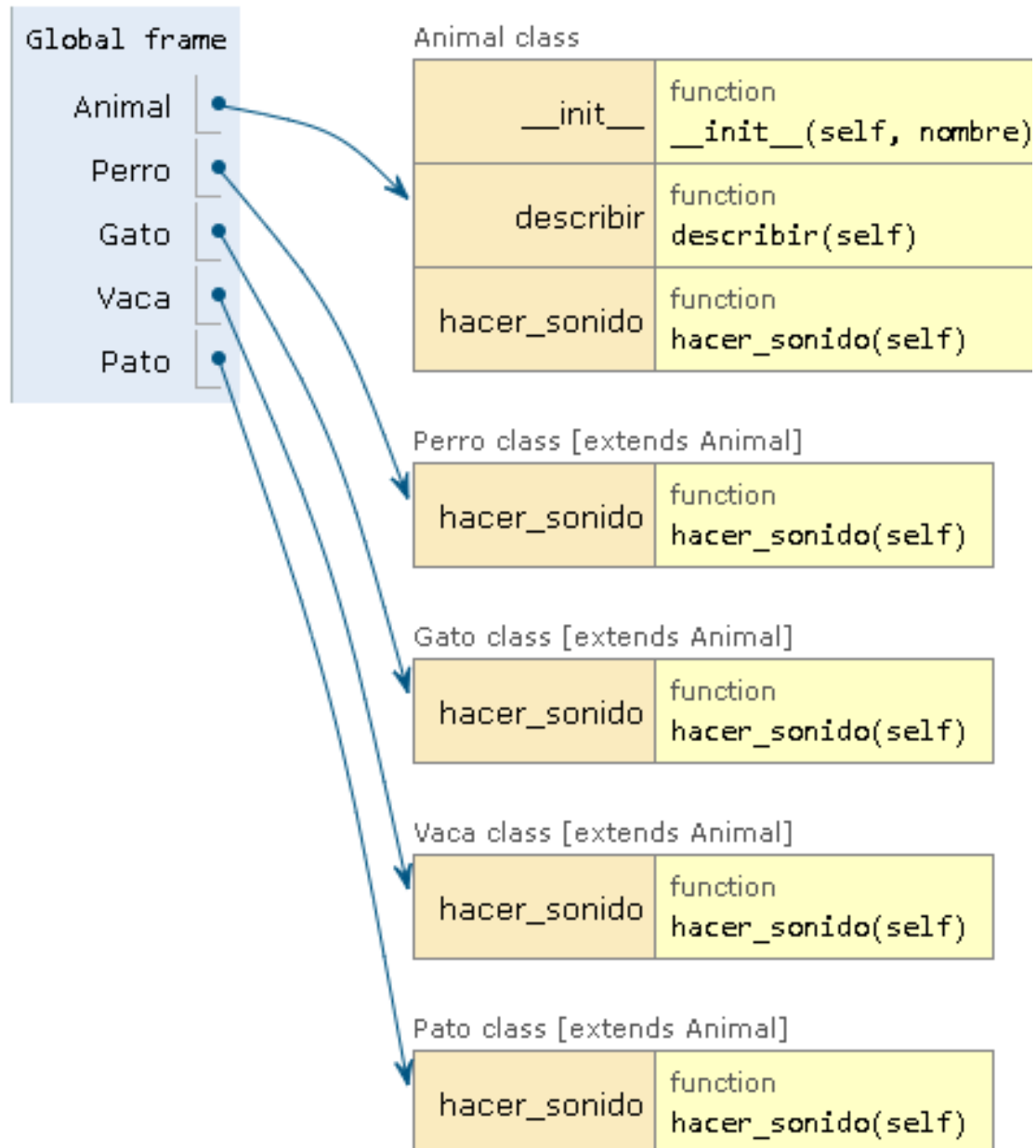
Programación Orientada a Objetos

Mgtr. Ramiro Leonardo Ramírez Coronel

16 de junio del 2026

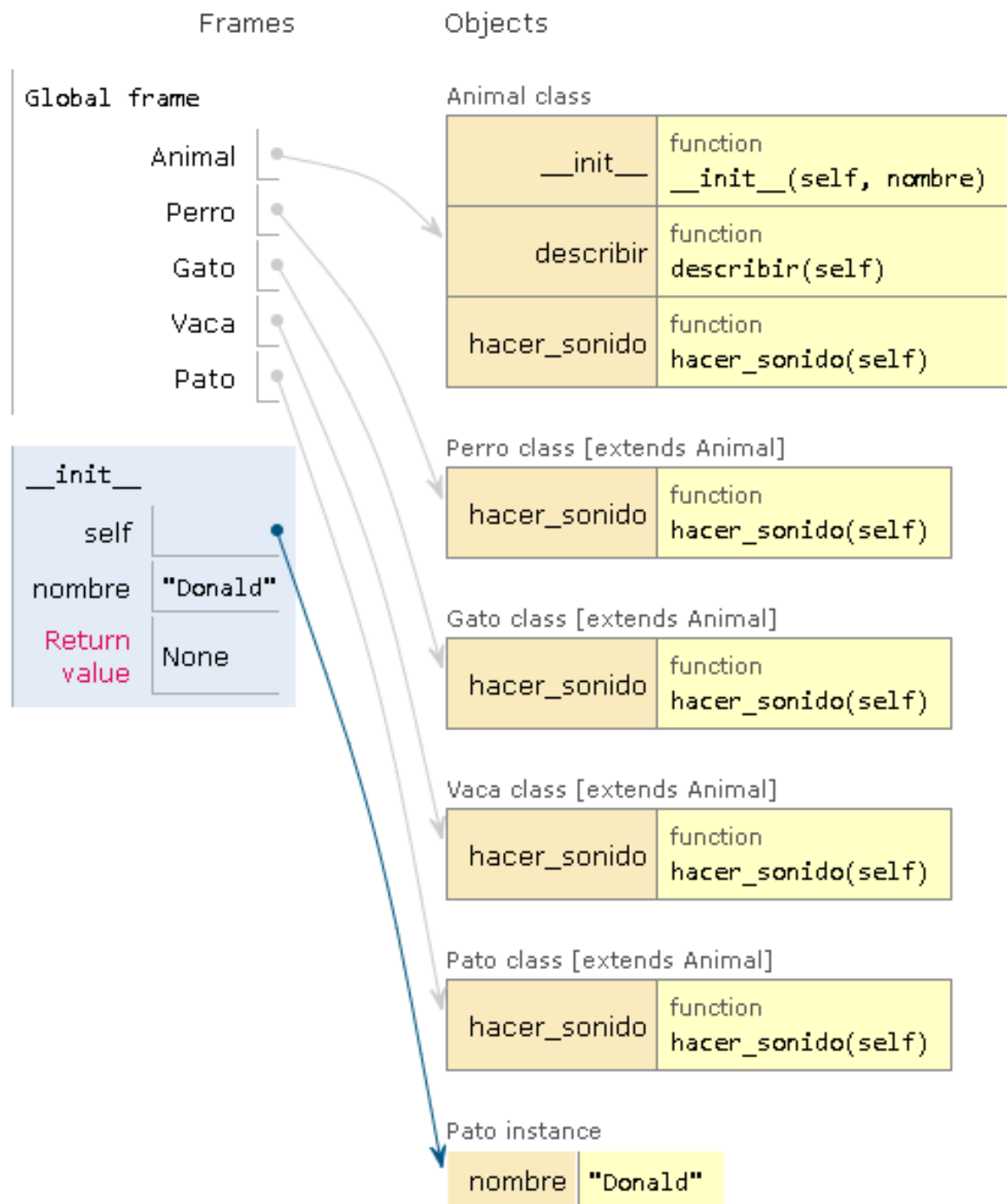
8 PASOS IMPORTANTES DENTRO DE UN CODIGO DE POLIMORFISMO

1.



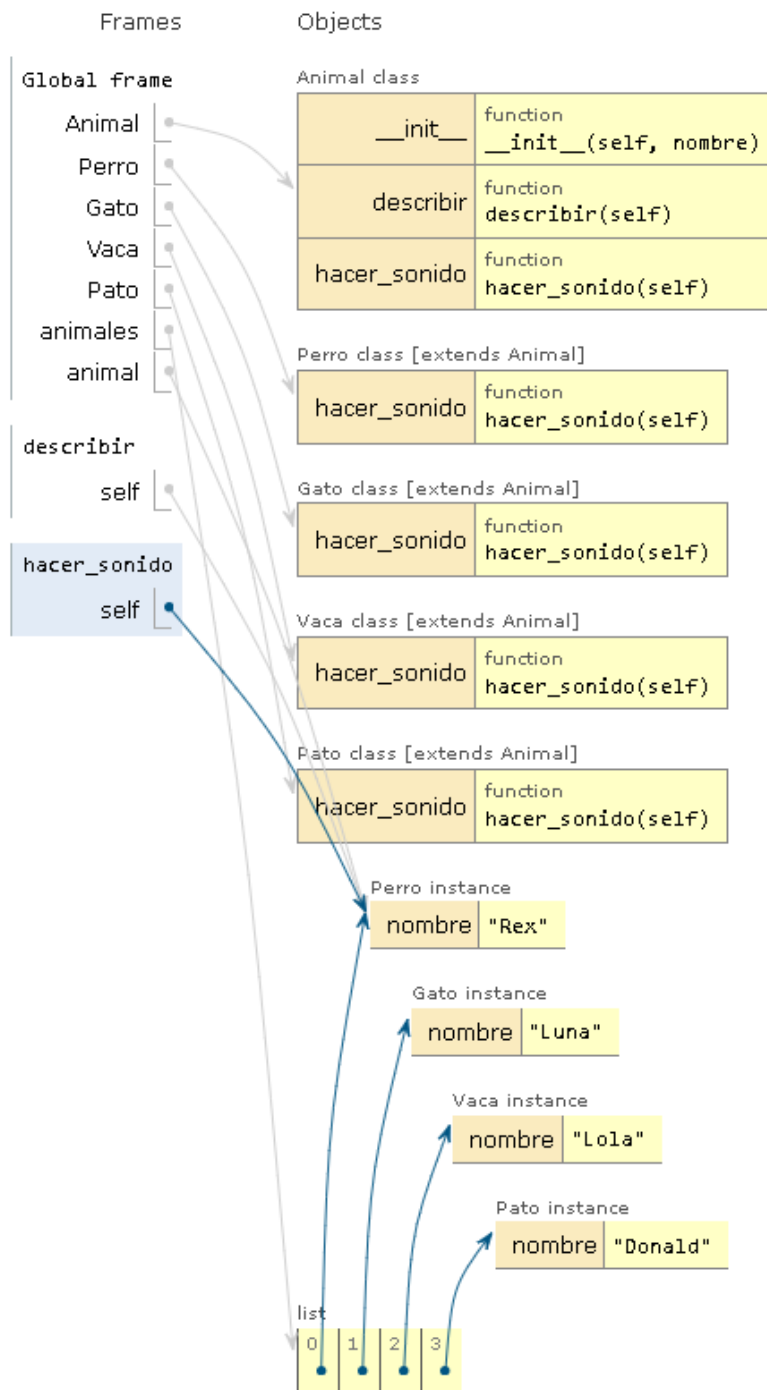
En esta primera parte, lo que está pasando es que Python lee las clases y las guarda en la memoria global para armar la estructura de herencia antes de crear los animales reales. Primero registra a la clase padre Animal con su constructor y sus métodos base, y abajo engancha a las clases hijas (Perro, Gato, Vaca y Pato) que por eso salen con el [extends Animal]; cada hijo se guarda casi vacío porque hereda todo lo del padre automáticamente, pero sí registran su propio método hacer_sonido para que después, cuando creamos los objetos en la lista, cada uno responda con su propio ruido aplicando polimorfismo.

2.



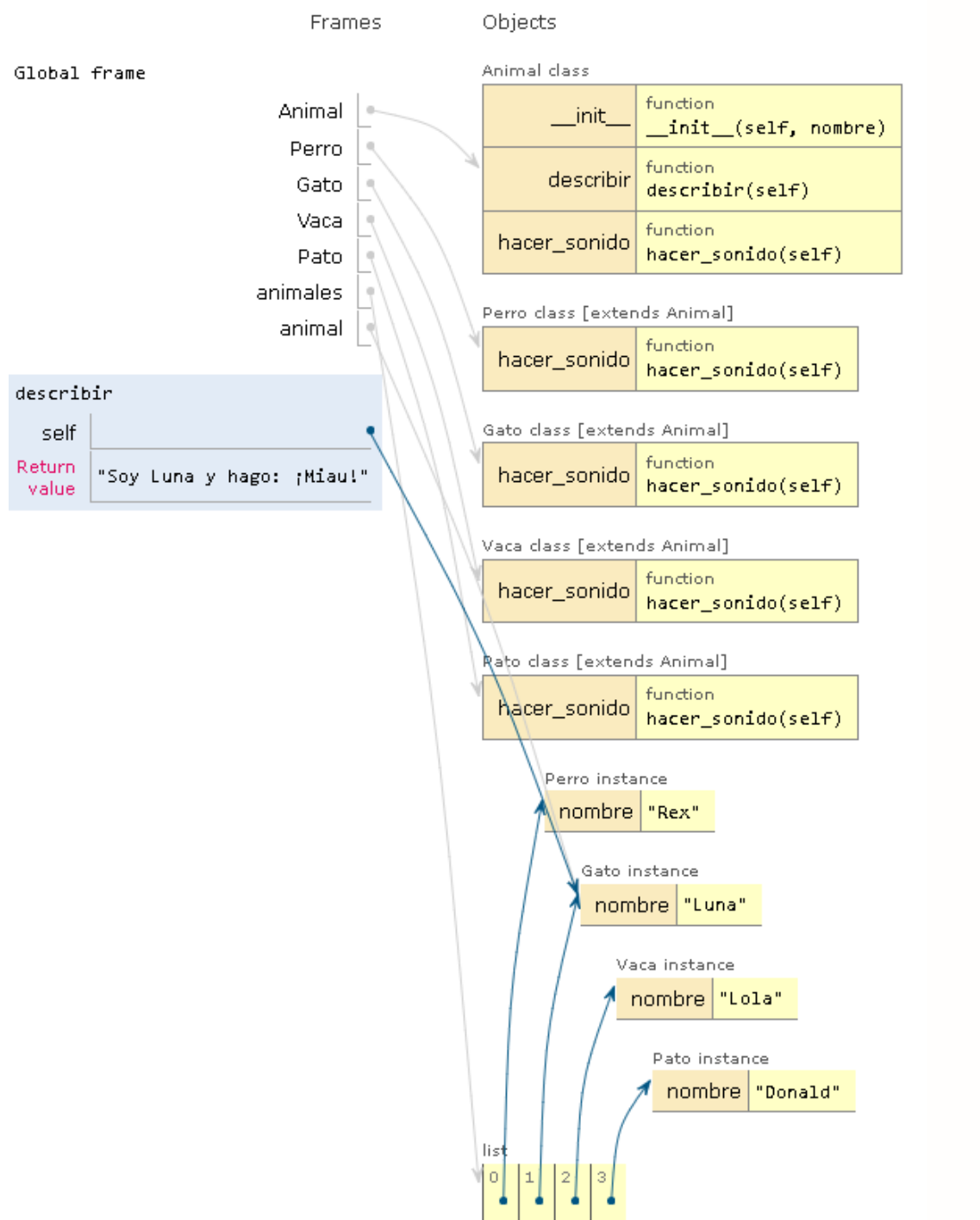
Aquí, en lo que sería la segunda parte, lo que está pasando es que el programa ya empezó a ejecutar las líneas de la lista de animales y se metió a crear el objeto de la clase `Pato`. Se abrió un bloque temporal en memoria (`__init__`) donde se recibe el parámetro `nombre` con el valor de `"Donald"`, y justo la flecha azul nos muestra cómo se termina de fabricar el objeto real llamado `Pato instance` abajo a la derecha, guardando de forma segura su atributo `nombre` y quedando vinculado a su clase para saber qué métodos puede usar.

3.

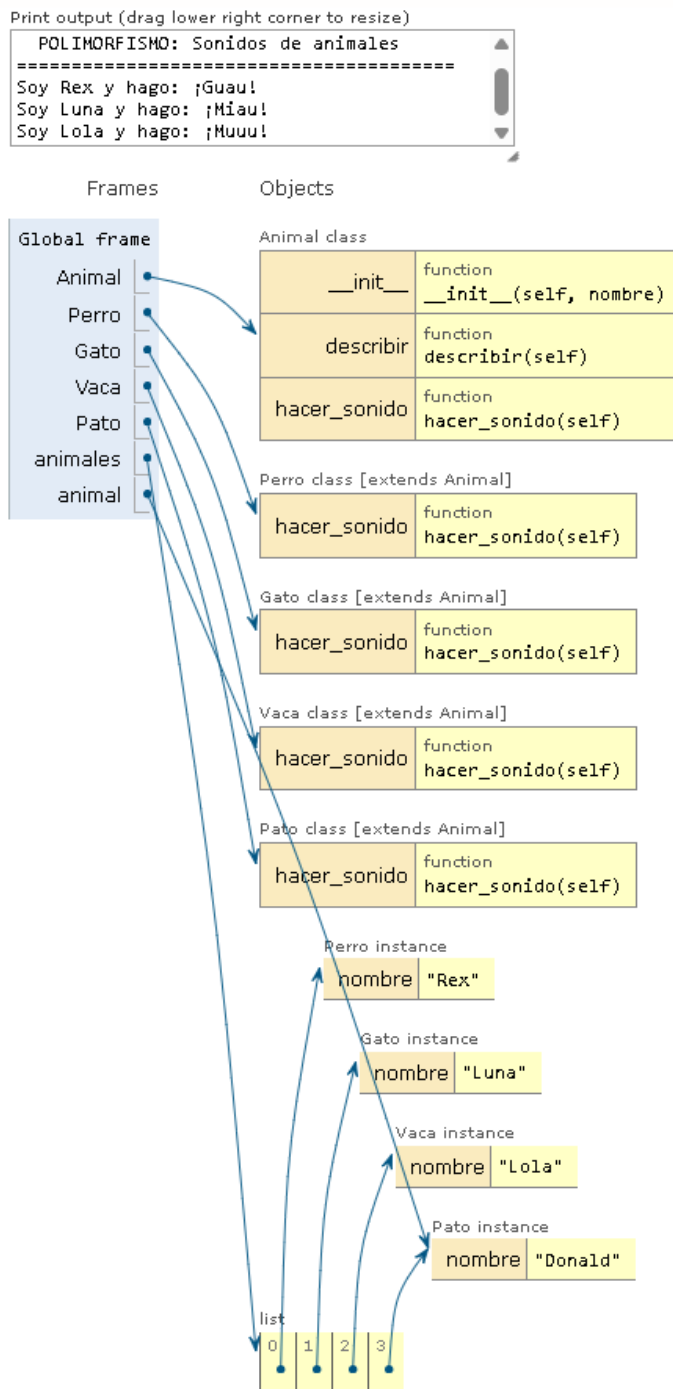


En esta tercera parte, lo que se ve es el polimorfismo en su punto clave: el programa empezó a recorrer la lista de objetos (`list`) y llamó al método `describir` para el primer elemento, que es el `Perro` instance (`"Rex"`). Como vemos en los bloques azules de la izquierda, se abrieron los espacios en memoria para `describir` y `hacer_sonido`, y la flecha azul apunta directo al bloque de la clase `Perro`; esto pasa porque, aunque todos los animales de la lista heredan de la misma base, el sistema busca automáticamente la versión sobrescrita de `hacer_sonido` que le corresponde a `Perro` para que este objeto responda con su propio comportamiento específico.

4.



En esta cuarta parte, el bucle avanzó al índice 1 de la lista y ahora es el turno del objeto Gato instance ("Luna"). En el bloque azul de la izquierda vemos que el método `describir` ya terminó de ejecutarse y el sistema generó un Return value que dice "Soy Luna y hago: ¡Miau!"; esto demuestra de forma clara el polimorfismo, ya que el programa usó de forma automática la versión de `hacer_sonido` de la clase Gato para armar el texto final, devolviendo el resultado correcto antes de limpiar ese bloque de la memoria y pasar al siguiente animal.

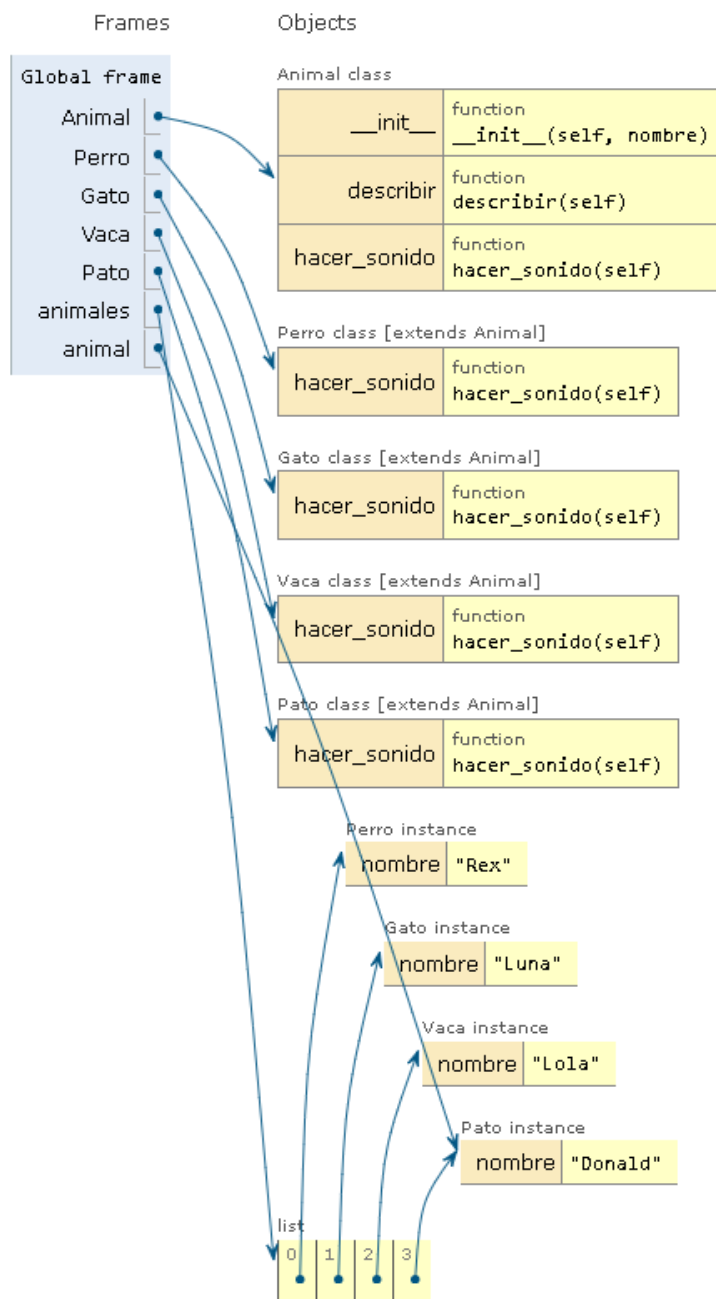


En esta quinta parte, el bucle avanzó al índice 2 de la lista para procesar al objeto Vaca instance ("Lola"), y arriba en el cuadro blanco de Print output ya podemos ver reflejado todo el historial impreso de las iteraciones anteriores. En esta captura los bloques azules de ejecución ya se cerraron, lo que significa que el método terminó de correr y mandó a la consola la línea "Soy Lola y hago: ¡Muuu!" usando de forma automática el `hacer_sonido` de la clase Vaca, dejando la lista limpia y lista para pasar al último elemento que es el pato.

6.

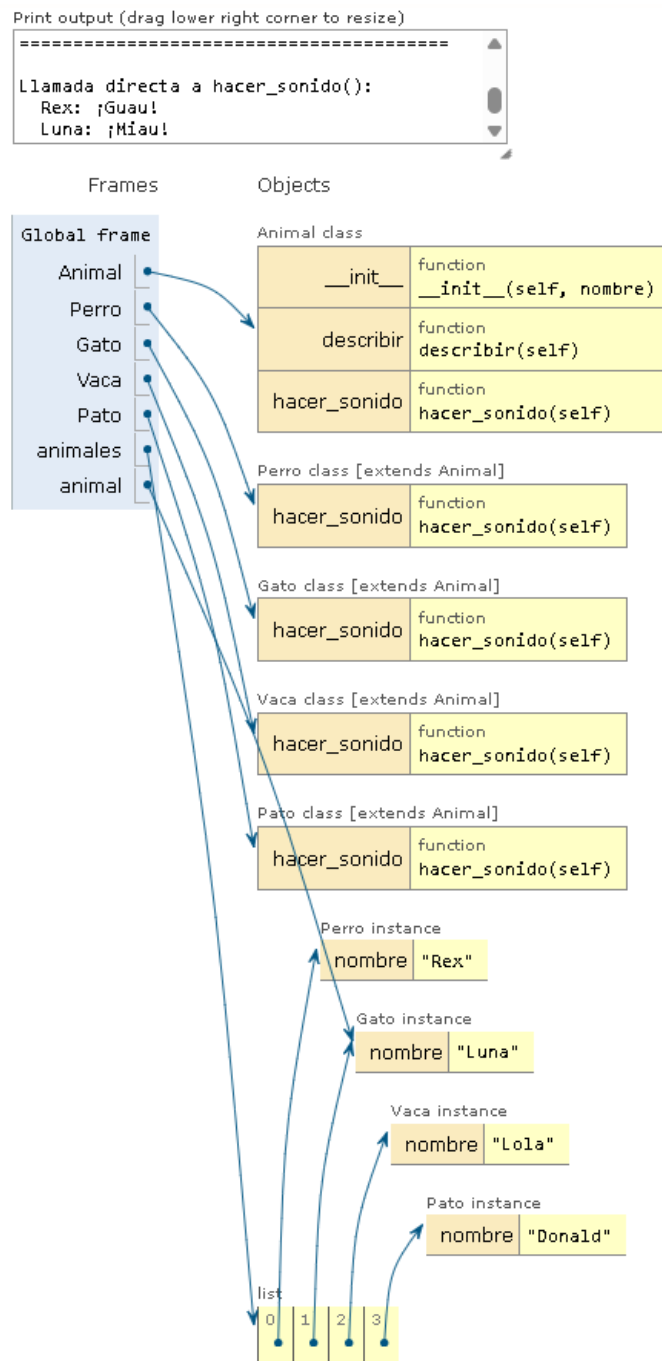
Print output (drag lower right corner to resize)

```
Soy Rex y hago: ¡Guau!
Soy Luna y hago: ¡Miau!
Soy Lola y hago: ¡Muuu!
Soy Donald y hago: ¡Cuac!
=====
```



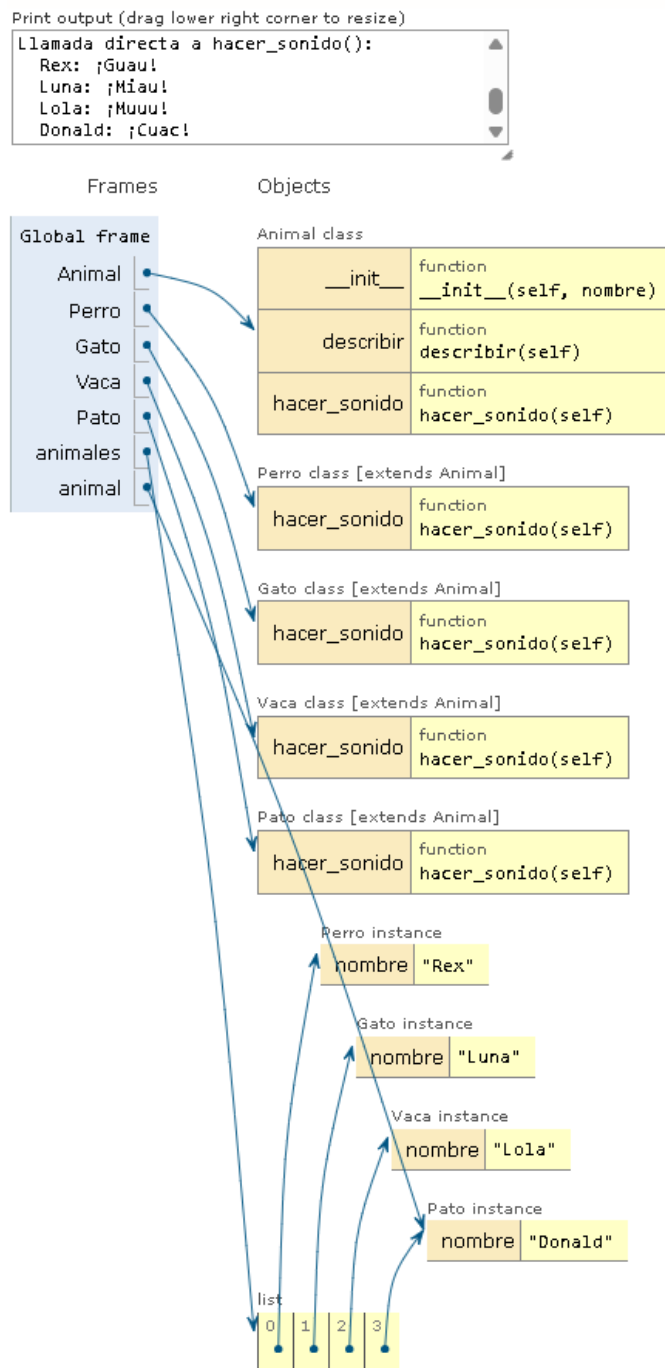
En esta sexta parte, el programa llega a la última iteración del bucle recorriendo el índice 3 de la lista, que corresponde al objeto Pato instance ("Donald"). Como se ve arriba en la pantalla de salida (Print output), el método `describir` ya terminó de ejecutarse y el programa imprimió la línea final "Soy Donald y hago: ¡Cuac!" tras enlazar automáticamente el comportamiento específico de la clase Pato, completando así el recorrido polimórfico de toda la lista y dejando la memoria global estable con todas las instancias y clases creadas.

7.



En esta séptima parte, el programa sale del bucle que recorría la lista completa y empieza una nueva sección en la consola llamada "Llamada directa a hacer_sonido()". Como se nota en la pantalla de salida (Print output), aquí ya no se usa el método intermedio describir, sino que el código invoca directamente la función hacer_sonido() desde los objetos individuales en memoria; primero lo hace con el Perro instance imprimiendo "Rex: ¡Guau!" y luego con el Gato instance imprimiendo "Luna: ¡Miau!", demostrando que los objetos mantienen el acceso directo a sus métodos sobrescritos en cualquier momento de la ejecución.

8.



En esta octava y última parte, el programa termina por completo la ejecución de todas sus líneas de código. Como se ve arriba en el Print output, se completaron las llamadas directas que faltaban para la Vaca instance ("Lola") y el Pato instance ("Donald"), imprimiendo sus ruidos correspondientes; al final, la memoria se queda estática mostrando todo el ciclo de vida del programa: las estructuras de las clases con sus herencias fijas, la lista que sirvió para agruparlos y los cuatro objetos independientes que se crearon, cerrando así la demostración práctica de cómo conviven la herencia, la sobreescritura de métodos y el polimorfismo en Python.